SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA1014	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 1 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	Arunprasath R	**Reg. No.:**	192524077
Date:	14/10/2026	**Duration:**	60 minutes

Code of Conduct

I Arunprasath R (192524077) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Arunprasath R

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario** ○ Carefully understand the given problem statement.
    - Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements** ○ List all functional requirements required by the proposed system.
    - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.

3. **Identify Actors and Use Cases** ○ Identify all primary and secondary actors interacting with the system. ○ List the major use cases associated with each actor.
    - Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)** Develop an SRS document containing: ○ Introduction ○ Problem Description
    - Scope of the System ○ Functional Requirements ○ Non-Functional Requirements ○ Use Case Descriptions ○ Data Requirements ○ External Interface Requirements ○ System Features
5. **Identify Assumptions and Constraints** ○ List the assumptions considered while developing the system.
    - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency** ○ Verify whether all stakeholder requirements have been addressed. ○ Check for ambiguity, redundancy, inconsistency, and missing requirements.
    - Suggest improvements to enhance the quality and completeness of the requirement specification.

Intelligent Academic Timetable Conflict Resolution Platform

Industry Context

Universities must schedule hundreds of courses, faculty members, classrooms, laboratories, and student batches. Manual timetable preparation can create conflicts and inefficient room utilization.

Problem Statement

Design and develop an intelligent timetable platform that automatically detects scheduling conflicts and generates feasible timetables based on faculty availability, classroom capacity, laboratory requirements, course combinations, and student batches.

Objectives

- Manage academic scheduling constraints.
- Detect faculty conflicts.
- Detect classroom conflicts.
- Handle laboratory requirements.
- Optimize room utilization.
- Support timetable modifications.

Expected Outcomes

- Conflict-free timetables.
- Reduced administrative effort.
- Better classroom utilization.
- Faster timetable generation.
- Improved academic scheduling.

SCENARIO BASED REQUIREMENT ANALYSIS

Intelligent Academic Timetable Conflict Resolution Platform

1.1 Purpose

The purpose of this project is to develop an Intelligent Academic Timetable Conflict Resolution Platform that automatically generates academic timetables by considering faculty availability, classroom capacity, laboratory requirements, courses, and student batches. The system identifies scheduling conflicts and helps administrators create efficient, conflict-free timetables.

1.2 Industry Domain:

The proposed system belongs to the Education and Academic Administration domain. It is designed for universities, colleges, and educational institutions that need to manage complex academic schedules.

1.3 Stakeholders:

The major stakeholders are:

Academic Administrators: Manage courses and generate timetables.

Faculty Members: Provide availability and view assigned schedules.

Students: View their class timetables.

Department Coordinators: Manage department-level scheduling.

Laboratory Coordinators: Manage laboratory availability and requirements.

System Administrator: Maintains the system and manages user access.

1.4 Existing Challenges:

- Traditional timetable preparation faces several challenges:
- Faculty members may be assigned to multiple classes at the same time.
- Two classes may be scheduled in the same classroom simultaneously.
- Laboratories may not be available when required.
- Classroom capacity may be insufficient for the assigned student batch.
- Manual scheduling requires considerable time and effort.
- Timetable modifications may introduce new conflicts.
- Classroom resources may be used inefficiently.

1.5 Objectives:

The main objectives are:

1. To automate academic timetable generation.

2. To identify faculty scheduling conflicts.

3. To prevent classroom booking conflicts.

4. To allocate laboratories according to course requirements.

5. To consider classroom capacity and student batch requirements.

6. To improve classroom utilization.

7. To support timetable modifications and updates.

8. To reduce administrative workload and scheduling time.

2. Problem Description:

Universities must coordinate numerous courses, faculty members, classrooms, laboratories, and student batches within a fixed academic schedule. Preparing timetables manually makes it difficult to satisfy all scheduling constraints simultaneously.

For example, a faculty member may be assigned to two classes during the same time slot, or two student batches may be allocated to the same classroom. Similarly, a practical session may be scheduled in a laboratory that is already occupied.

These problems can cause timetable conflicts, delays, inefficient resource utilization, and inconvenience to students and faculty members.

The proposed Intelligent Academic Timetable Conflict Resolution Platform addresses these issues by collecting academic scheduling information, applying scheduling constraints, detecting conflicts, and generating feasible timetables.

The system also supports timetable modifications and checks whether the proposed changes introduce new conflicts.

3. Scope of the System

The system focuses on automating and improving academic timetable preparation.

3.1 In-Scope Features

- Faculty information management.
- Course and subject management.
- Student batch management.
- Classroom and laboratory management.
- Faculty availability management.

- Automatic timetable generation.
- Faculty conflict detection.
- Classroom conflict detection.
- Laboratory allocation.
- Classroom capacity validation.
- Student batch scheduling validation.
- Room utilization optimization.
- Timetable modification and regeneration.
- Timetable viewing and downloading.
- Conflict reports and scheduling summaries.

3.2 Expected Outcomes

The system is expected to provide:

- Conflict-free timetables that satisfy the defined constraints.
- Reduced administrative effort.
- Improved classroom and laboratory utilization.
- Faster timetable generation.
- Easier timetable modification.
- Better coordination among administrators, faculty members, and students.

5. Identification of Actors and Use Cases

5.1 Primary Actors

Primary actors directly interact with the system to achieve their goals.

1. Academic Administrator

- Manages faculty, courses, and student batches.
- Manages classrooms and laboratories.
- Generates academic timetables.
- Detects and resolves conflicts.
- Modifies and publishes timetables.
- Generates reports.

2. Department Coordinator

- Provides department scheduling requirements.
- Updates course and faculty availability information.
- Reviews generated timetables.
- Requests timetable modifications.

3. Faculty Member

- Provides availability information.
- Views assigned timetables.
- Reports scheduling problems.

4. Student

Views the published timetable for their batch.

Downloads the timetable when supported.

5. Laboratory Coordinator

Updates laboratory availability.

Maintains laboratory facilities.

Verifies laboratory allocations.

5.2 Secondary Actors

1. System Administrator

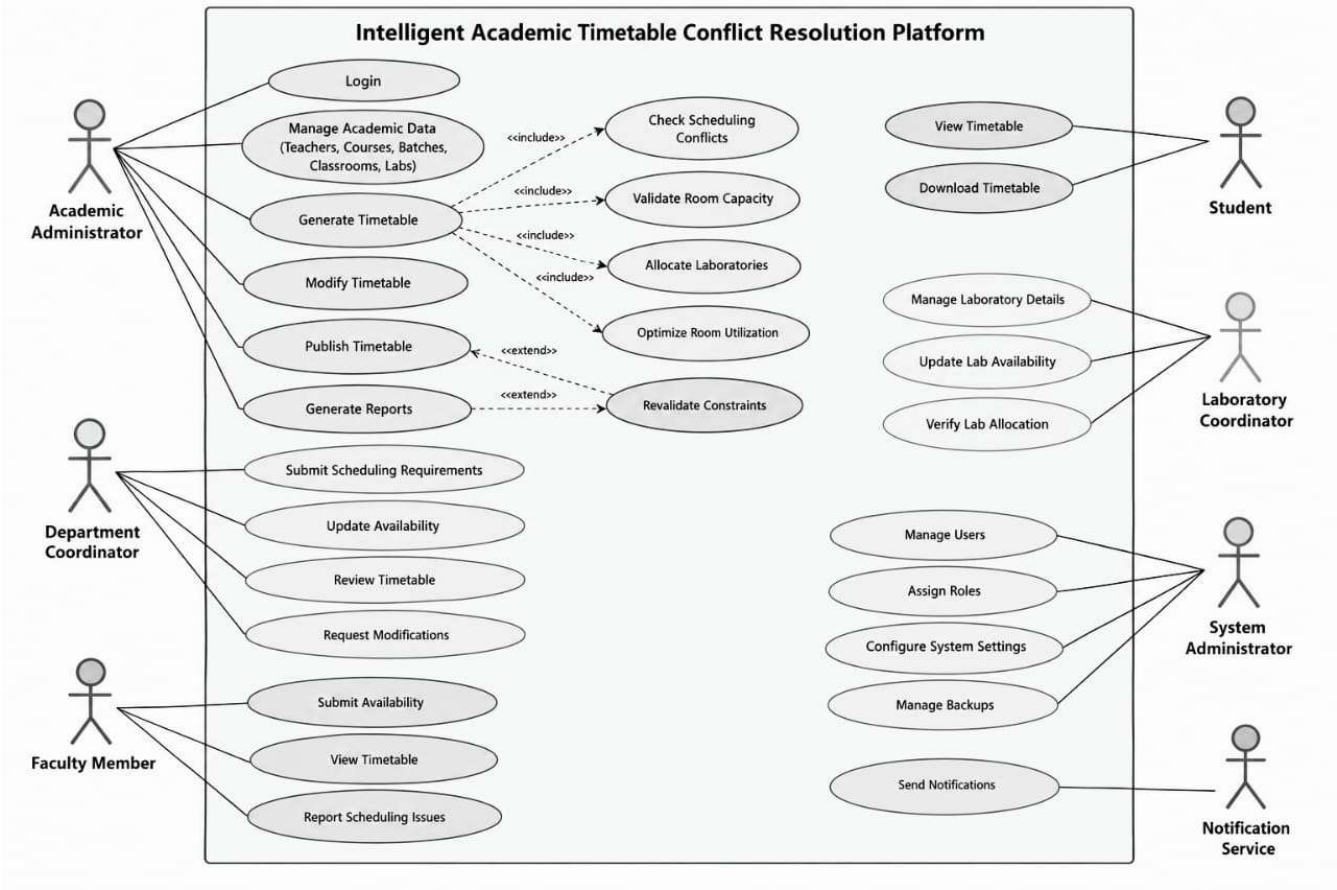
- Manages user accounts and permissions.
- Maintains system settings.
- Monitors system operation and backups.

2. Notification Service

Sends timetable publication and modification notifications when notification integration is enabled.

6. Use Case Diagram

The following diagram represents the interactions between the actors and the Intelligent Academic Timetable Conflict Resolution Platform.



7. Use Case Descriptions

7.1 Use Case 1: Generate TimetableUse

Use Case Name	Generate Timetable
Primary Actor	Academic Administrator
Objective	Generate a feasible academic timetable automatically.
Preconditions	Academic data, scheduling constraints, and resource availability must be available.
Trigger	The administrator selects the Generate Timetable option.

Main Flow

1. The administrator logs into the system.

2. The administrator selects the academic semester and department.

3. The system retrieves course, faculty, classroom, laboratory, and student batch information.

4. The system checks the available time slots.

5. The scheduling engine allocates courses to suitable time slots and resources.

6. The system checks faculty, classroom, laboratory, and student batch conflicts.

7. The system evaluates the generated timetable against the defined constraints.

8. The system displays the generated timetable.

9. The administrator reviews and approves the timetable.

10. The system saves the approved timetable.

Field	Description
Use Case ID	UC-02
Use Case Name	Detect Scheduling Conflicts
Primary Actor	Academic Administrator
Objective	Identify overlapping or invalid timetable allocations.
Preconditions	A timetable draft or proposed timetable modification exists.
Trigger	table is generated, modified, or submitted for validation

Main Flow

1. The system retrieves the timetable entries.

2. It checks whether any faculty member is assigned to overlapping classes.

3. It checks whether any classroom is allocated to multiple classes simultaneously.

4. It checks student batch overlaps.

5. It verifies classroom capacity and laboratory requirements.

6. It identifies unavailable time slots and invalid allocations.

7. It displays detected conflicts and their details.

8. The administrator reviews the conflicts and makes corrections.

8. Data Requirements

8.1 Data Validation Rules

1. Every timetable entry must reference a valid course, faculty member, student batch, and time slot.

2. A classroom must have sufficient capacity for the assigned student batch.

3. A laboratory session must be assigned to a suitable laboratory.

4. A faculty member cannot teach two classes at the same time

5. A classroom cannot host two classes simultaneously.

6. A student batch cannot attend two classes simultaneously.

7. A scheduled session must fit within the available time slot.

8. Published timetable changes must be recorded for traceability.

8.2 Data Storage

The system may use a relational database such as MySQL or PostgreSQL to store academic data. Appropriate primary keys, foreign keys, and validation rules should be used to maintain data integrity.

9. External Interface Requirements

9.1 User Interface Requirements

The system shall provide:

- A login page for authorized users.An administrator dashboard.
- Forms for managing faculty, courses, classrooms, and laboratories.
- A timetable generation page.
- A timetable calendar or grid view.
- A conflict notification panel.
- A timetable modification interface.
- A reporting page
- A student and faculty timetable viewing page.

9.2 Hardware Interface Requirements

The system requires:

- A computer or mobile device for users.
- A server or cloud-hosted environment to run the application.
- Network connectivity for accessing the web application.
- No specialized hardware is required.

9.3 Software Interface Requirements

The system may interact with

- A relational database for storing scheduling information.
- A web browser for displaying the user interface.
- An authentication component for access control.
- A timetable scheduling engine for resource allocation.
- An optional email or notification service for timetable updates.

10. System Features

10.1 Automated Timetable Generation

The system generates timetables based on course requirements, faculty availability, room availability, student batches, and institutional scheduling rules.

10.2 Faculty Conflict Detection

The system checks whether a faculty member is assigned to overlapping sessions and prevents invalid assignments.

10.3 Classroom Conflict Detection

The system verifies classroom availability and prevents multiple classes from using the same classroom at the same time.

10.4 Laboratory Allocation

The system assigns practical sessions to suitable laboratories according to capacity, availability, and required facilities.

10.5 Classroom Capacity Validation

The system checks classroom seating capacity before assigning a student batch.

10.6 Room Utilization Optimization

The system attempts to reduce unnecessary room allocation, improve the use of available classrooms, and avoid unsuitable resource assignments.

10.7 Timetable Modification

The system allows authorized administrators to change schedules and validates the proposed changes before saving them.

10.8 Conflict Reporting

The system displays conflict types, affected resources, and relevant timetable entries to help administrators resolve scheduling problems.

10.9 Timetable Publication

The system allows administrators to publish approved timetables and make the latest version available to students and faculty members.

11. Assumptions and Constraints

11.1 Assumptions

- The university provides accurate information about courses, faculty, student batches, and classrooms.
- Faculty members provide their availability before timetable generation.
- Classroom capacities and laboratory facilities are correctly recorded.
- The institution defines the academic working days and time slots.
- The administrator has authority to approve and publish timetables.
- All mandatory scheduling constraints are clearly defined.
- Users have access to a suitable device and network connection.
- Timetable generation uses the latest approved academic data.
- The institution provides a method for reporting changes to faculty availability and resource availability.

11.2 Technical Constraints

The system must operate within the capabilities of its selected programming language, database, and hosting environment.

Timetable generation may become computationally expensive as the number of courses and constraints increases.

Internet connectivity is required for remote access to the hosted system.

The system must protect stored academic information from unauthorized access.

11.3 Operational Constraints

- Administrators must enter or import accurate scheduling data.
- Faculty availability and room information must be updated regularly.
- The system must follow institutional timetable policies.
- Timetable publication requires appropriate administrative approval.

11.4 Economic Constraints

The project should use available institutional resources and cost-effective technologies.

Hosting, database maintenance, backups, and optional notification services may incur costs.

Development and maintenance should remain within the approved project budget.

11.5 Legal Constraints

Personal information must be handled according to applicable privacy and data protection requirements.

User access must be controlled to prevent unauthorized disclosure of faculty and student information.

The system must follow applicable institutional policies for storing and sharing academic data.

11.6 Time Constraints

The project must be developed within the assigned academic project period.

Requirement analysis, design, implementation, testing, and documentation must be completed within the available time.

Advanced optimization and integration features may be deferred if time is limited.

12. Requirement Validation: Completeness and Consistency

Requirement validation ensures that the SRS accurately describes the system's intended behavior and addresses stakeholder needs.

12.1 Ambiguity Check

Requirements such as "fast timetable generation" and "better room utilization" may be interpreted differently by different stakeholders.

Improvement: Define measurable targets, such as generating a department timetable within 30 seconds under specified test conditions and measuring classroom utilization before and after optimization.

12.2 Redundancy Check

Faculty conflict detection and classroom conflict detection are separate requirements because they validate different resources. However, general conflict checking and conflict rechecking may overlap.

Improvement: Define a common validation component that checks all scheduling constraints whenever a timetable is generated or modified.

12.3 Consistency Check

Timetable generation, modification, and publication must follow the same mandatory scheduling rules.

Improvement: The system must validate the timetable before publication and repeat validation after every relevant modification.

12.4 Missing Requirement Check

- The initial requirements may not fully address:
- Maximum teaching hours per faculty member.
- Maximum consecutive classes.
- Required breaks between sessions.
- Special classroom equipment requirements.
- Institutional holidays and non-working days.
- Preferences for morning or afternoon sessions.
- Timetable version history and rollback.
- Notifications when published timetables change.
- Identification of unsatisfied constraints when no feasible timetable exists.
- Improvement: Add these requirements when they are applicable to the institution's scheduling policies.

12.5 Feasibility Validation

- The system must distinguish between mandatory constraints and preferences.
- Mandatory constraints include:
- No faculty member can teach two classes simultaneously.
- No classroom can host two classes simultaneously.
- A student batch cannot have overlapping classes.
- Room capacity must satisfy the batch size.
- Laboratory requirements must be met.
- Preferences may include:
- Minimizing gaps between classes.
- Reducing unnecessary classroom changes.
- Distributing faculty workloads more evenly.
- Improving room utilization.
- The system must satisfy all mandatory constraints before declaring a timetable feasible. Preferences should be optimized where possible without violating mandatory constraints.

12.6 Suggested Improvements

The following improvements can enhance the quality of the SRS:

- Define measurable performance targets.
- Separate mandatory constraints from scheduling preferences.
- Specify the expected number of courses, faculty members, and rooms.
- Define the procedure for handling unsatisfiable scheduling constraints.
- Include timetable versioning and change history.
- Specify backup frequency and recovery expectations.
- Define user roles and permissions clearly.

- Prepare test cases for each functional requirement.
- Validate requirements with administrators, faculty members, and student representatives.
- Maintain traceability between stakeholder needs, functional requirements, and test cases.

Conclusion

The Intelligent Academic Timetable Conflict Resolution Platform is designed to automate academic scheduling and reduce the difficulties associated with manual timetable preparation.

The system manages courses, faculty members, student batches, classrooms, and laboratories while detecting scheduling conflicts and validating resource availability. It supports automatic timetable generation, room utilization optimization, timetable modification, and conflict reporting.

The proposed SRS identifies the system's functional and non-functional requirements, stakeholders, actors, use cases, data requirements, interfaces, assumptions, and constraints. Requirement validation further improves the completeness, consistency, and clarity of the specification.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.

Software Requirement Specification (SRS) Documentation (15%)	SRS is well-structured, complete, professionally organized, and includes all required sections.	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	**Mark**
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25